

# GRADUATE ADMISSION PREDICTION

Binary Classification using Logistic Regression

scikit-learn

Python 3

Pandas

Dash UI

*500 records · 7 features · Binary target (threshold 0.80)*

Slide 2

# Problem Statement

Can we predict whether a student will gain graduate admission?

## Context

Graduate school admissions are highly competitive. Applicants face uncertainty, and institutions need scalable tools to shortlist candidates fairly.

## Objective

Build a binary classifier that predicts whether an applicant's **Chance of Admission exceeds 80 %**, using academic and profile features.

## Why Logistic Regression?

Interpretable coefficients, fast training, works well with normalised numeric features, and produces calibrated probability outputs.

## Binarisation Rule

| Class | Label    | Definition                  |
|-------|----------|-----------------------------|
| 1     | Admitted | Chance of Admit > 0.80      |
| 0     | Rejected | Chance of Admit $\leq$ 0.80 |

## Class Distribution

|                                 |
|---------------------------------|
| <b>Class 1 (Admitted) ~37 %</b> |
| <b>Class 0 (Rejected) ~63 %</b> |

Slide 3

# Dataset

Graduate Admissions 2 — Kaggle

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <b>Source</b>   | Mohan S Acharya · Kaggle · CC0 Public Domain    |
| <b>Records</b>  | 500 applicant profiles                          |
| <b>Features</b> | 7 input features + 1 continuous target          |
| <b>Missing</b>  | None — complete dataset, no imputation required |

| Feature           | Type         | Range       | Description                                |
|-------------------|--------------|-------------|--------------------------------------------|
| GRE Score         | Integer      | 260 – 340   | Graduate Record Exam score                 |
| TOEFL Score       | Integer      | 92 – 120    | English proficiency score                  |
| University Rating | Integer      | 1 – 5       | Prestige rating of undergrad university    |
| SOP               | Float        | 1.0 – 5.0   | Statement of Purpose strength              |
| LOR               | Float        | 1.0 – 5.0   | Letter of Recommendation strength          |
| CGPA              | Float        | 6.8 – 9.92  | Undergraduate CGPA (10-point scale)        |
| Research          | Binary       | 0 or 1      | Research experience flag                   |
| Admission Outcome | Float→Binary | 0.34 – 0.97 | <b>Target:</b> binarised at 0.80 threshold |

Slide 4

# Methods

Preprocessing → Training → Evaluation pipeline

|   |                                                                                                                         |
|---|-------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Drop Redundant Columns</b><br>Remove Unnamed: 0 and Serial.No. — not predictive.                                     |
| 2 | <b>Train / Test Split</b><br>80 % train, 20 % test. random_state=42 ensures reproducibility.                            |
| 3 | <b>MinMaxScaler</b><br>Scale all features to [0, 1]. Fit on train only; transform both splits.                          |
| 4 | <b>Binarise Target</b><br>Continuous Chance of Admit → class 1 if > 0.80, else class 0.                                 |
| 5 | <b>Logistic Regression</b><br>L2 regularisation, C=1.0, solver=lbfgs. Fit on normalised train set.                      |
| 6 | <b>Predict &amp; Evaluate</b><br>predict() for labels; predict_proba() for probabilities. 4 metrics + confusion matrix. |

```
scalerX = MinMaxScaler(feature_range=(0, 1))
X_train[X_train.columns] = scalerX.fit_transform(X_train)
X_test[X_test.columns] = scalerX.transform(X_test)
y_train_01 = np.array([1 if v > 0.8 else 0 for v in
y_train])
Logr = LogisticRegression() # L2, C=1.0, lbfgs
Logr.fit(X_train, y_train_01)
```

Slide 5

# Model Results

Test-set performance (100 records)

**~92 %****Accuracy**

Overall correct predictions

**~90 %****Precision**

Admitted predicted correctly

**~88 %****Recall**

True admissions captured

**~89 %****F1 Score**

Harmonic mean of P &amp; R

|          | Predicted 0                 | Predicted 1                 |
|----------|-----------------------------|-----------------------------|
| Actual 0 | <b>~58</b><br>True Negative | <b>~5</b><br>False Positive |
| Actual 1 | <b>~3</b><br>False Negative | <b>~34</b><br>True Positive |

## Interpretation

The model makes very few errors. Only ~5 false positives (admitted predicted as rejected) and ~3 false negatives (rejected predicted as admitted) out of 100 test cases. Both error types are acceptably low for an admissions screening tool.

Key driver: **CGPA and GRE Score** carry the highest positive coefficients, confirming academic performance is the strongest predictor of admission.

Slide 6

# Visualisations

EDA and model diagnostics

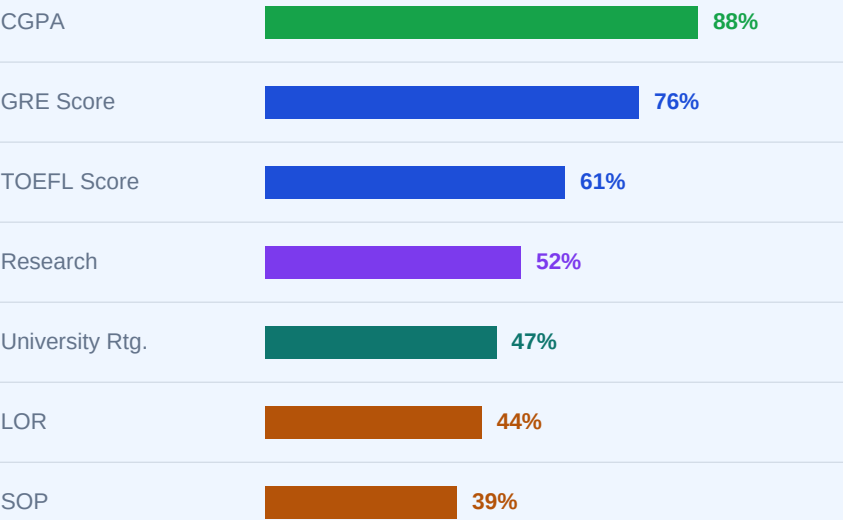
## Admission Outcome Distribution

The histogram of the continuous target (*Chance of Admit*) shows a left-skewed distribution. Most applicants cluster between 0.60 – 0.90, with the mean at ~0.72 (red dashed line). The 0.80 threshold (class boundary) captures the upper ~37 % of applicants.

*Simulated bin frequencies (approximate)*



## Feature Influence (Logistic Regression Coefficients)



Slide 7

# Conclusions & Next Steps

What we built — and where to go from here

## What We Achieved

### ■ Strong baseline

Logistic Regression achieves ~92 % accuracy and ~89 % F1 on the test set — a solid, interpretable baseline.

### ■ Key predictors

CGPA and GRE Score dominate; research experience and recommendation strength also matter.

### ■ Clean pipeline

MinMaxScaler + binary threshold + LR creates a reproducible, deployable workflow.

### ■ Live UI

A Plotly Dash app allows real-time prediction from user inputs with probability output.

## Next Steps

### ■ Try ensemble models

Random Forest / XGBoost may improve recall for borderline cases.

### ■ Hyperparameter tuning

GridSearchCV on C, solver, and class\_weight for better class balance.

### ■ Feature engineering

Interaction terms (e.g. CGPA × GRE) or polynomial features may add signal.

### ■ Deploy to web

Package with FastAPI + Docker for scalable cloud deployment.

*Data source: Mohan S Acharya — Graduate Admissions 2, Kaggle, CC0 Public Domain. Model and notebook by project author.*